

# Project 5 – Hidden Markov Model and Markov Decision Process (200 pts)

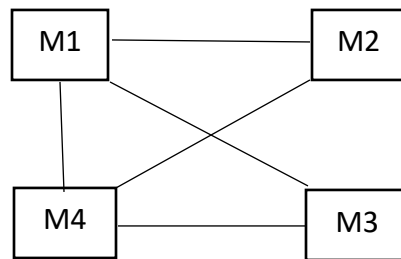
## Task 1 – Hidden Markov Model (100 pts)

In this task, you will apply **Hidden Markov Models (HMMs)** to a cybersecurity scenario. Modern networks are susceptible to attackers who can move stealthily between connected machines. Each machine is equipped with an **Intrusion Detection System (IDS)** that may trigger alarms if an attack occurs.

Your **goal** is to help a security engineer:

1. **Replay the attacking path** – infer the most likely sequence of machines visited by the attacker given a sequence of observed alarm triggering
2. **Estimate the current attacker location** – determine which machine is most likely being attacked at the current time.
3. **Predict near-future attacks** – forecast which machines the attacker may visit in the next few time steps.

**Limitations:** this task will give you hands-on experience with HMM inference algorithms, including **Viterbi algorithm**, **filtering**, and **prediction**, in a simplified network security context. Be aware that the real-world attacks and defenses are much more complex than the simplified model used here.



**An example network topology with 4 machines (M1~M4). Each with an alarm (IDS) installed (A1~A4).**

- Inputs and outputs

**Transition model:** from each machine, the attacker is equally likely to hop to other directly connected machines. In the language of HMM, each hop makes a state transition  $X_i \rightarrow X_{i+1}$ . For example, in the above network, if the attacker is currently on M3, he/she will have 0.5 probability to hop to M1, and 0.5 to M4 as the next machine. We can assume that the attacker chooses to keep hopping between machines to avoid being detected.

**Sensor model:** we assume a noisy sensor model here. Each IDS alarm has an independent probability of being triggered. If the attacker is currently on a machine, the alarm on this machine will have a higher probability to be triggered (true positive). For alarms not on this machine, they may give false positive warnings with a smaller probability. As the evidence of each time step, the status of all alarms must be included. For example, in the previous network, assume at time step 1, the attacker is on machine M2, i.e., the state of the HMM  $X_1=M_2$ , then the evidence  $e_1$  might be {A1: 0, A2: 1, A3: 1, A4: 0}, which means that A2 gives a true positive warning and A3 is somehow also triggered and gives a false positive warning. Assume the true positive probability is 0.9 and false positive probability is 0.2 for all alarms, then as an example of the sensor model, the probability of seeing this evidence:

$$\begin{aligned} P(e_1|X_1) &= P(A1 = 0, A2 = 1, A3 = 1, A4 = 0|X_1 = M_2) = P(A1 = 0|X_1 = M_2) * P(A2 = 1|X_1 = M_2) * P(A3 = 1|X_1 = M_2) * P(A4 = 0|X_1 = M_2) \\ &= (1 - 0.2) * 0.9 * 0.2 * (1 - 0.2) = 0.1152 \end{aligned}$$

**Initial state distribution:** we will assume the attacker is equally likely to be on any machine at time 0, i.e.  $P(X_0 = M_i) = \frac{1}{\text{number of machines}}$

- **Starter code**

To help you write the program, a starter code `hmm.py` is provided. It provides two functions to create inputs to your program.

The function `network_generator(n=10, p=0.1)` is used to generate a randomly structured network as a connected graph. The output of this function is a dictionary to describe how machines are connected to one another in the network.

The other function `input_generator(graph, p1, p2, N)` is used to generate the ground-truth state sequence  $X_{0:N}$ , which is the actual attack path. Also generated is the associated evidence sequence  $e_{1:N}$ .

Please read the comments in the starter code on how to use them.

- **Your programs**

**Inference 1:** use the Viterbi's algorithm to find the most likely attack path given the evidence sequence  $e_{1:N}$ . You need to compare your inferred sequence with the ground truth path in python plots and provide analysis. Viterbi's algorithm can be found on page 15-18 of the lecture slides of topic 10. Be careful about value underflow issues when you implement this algorithm.

**Inference 2:** use the filtering algorithm to find the distribution  $P(x_N | e_{1:N})$ , and indicate which machine is the most likely one the attacker is currently attacking. You can find the matrix form of the filtering algorithm on page 25 of the lecture slides of topic 10. Remember to normalize your forward message f.

**Inference 3:** use prediction algorithm to find the distributions  $P(x_{N+d} | e_{1:N})$ , where  $d = 1, 2, 3, 4$ , and  $5$  must be computed by your program. Use each distribution to predict the most likely machine that is being attacked. Follow the hints on the starter code to examine the quality of your prediction. You can find the prediction algorithm on page 8 of the lecture slides of topic 10.

- **Evaluation requirements**

A group of control parameters are provided in the starter code

| Parameter                 | Interpretation                                                                                                                                                                                                                 |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| nMachines=5               | The Number of machines in the network is 5.<br>The domain size of every state variable $X_i$ is 5.                                                                                                                             |
| connectionDensity = 0.2   | The probability of connecting any 2 unconnected machines in the network is 0.2.                                                                                                                                                |
| alarmRateAttackOn = 0.95  | Alarm triggering probability = 0.95 under true positive                                                                                                                                                                        |
| alarmRateAttackOff = 0.05 | Alarm triggering probability = 0.05 under false positive                                                                                                                                                                       |
| nObservations = 20        | Make a sequence of 20 observations, i.e. $N = 20$ .<br>The evidence sequence generated by the provided function will be $e_{1:20}$ .<br>The ground-truth attack sequence generated by the provided function will be $X_{0:20}$ |

You should first finish your program and use the above parameters to test correctness. Next, tune some of the parameters to thoroughly test your program's robustness and limitations. For instance, the two alarmRate parameters directly control the alarm's sensitivity and noise level. When they are changed, your program is supposed to behave differently. Please include your discovery in the report. Note that, each setting of these parameters should be tested with multiple executions to provide an average evaluation.

- **Submission requirements**

(1) Your program hmm.py

(2) A report that covers the following contents

- How you implemented each function and what data structures were used for convenience and efficiency
- With the basic setting, how accurate are your inference results compared with the ground truth? Is the accuracy stable across multiple executions? If there is inconsistency, where does it often happen? What might be the reason? You must provide at least two

plots for this part. One with perfect (or nearly perfect) inference, and the other with a few inconsistencies.

- The plots of inference accuracy versus tuned control parameters. Comment on how number of machines, alarm sensitivity and noise, and the length of sequence affect the quality of your inference
- How do you evaluate the advantage and limitation of HMM's inference?

## Task 2 – Markov Decision Process (100 pts)

In this task, you will help a cloud service provider to make sequential decisions by using a Markov decision process (MDP). We covered MDP on page 42-61 of the slides for topic 11.

The cloud company periodically monitors the use status of its cloud servers and adjusts the scale of its hardware accordingly to balance between revenue and cost. According to the workload, the use status of all the servers can be classified into 4 states:

$S = \{\text{"Low"}, \text{"Medium"}, \text{"High"}, \text{"Overloaded"}\}$  # these will be the 4 states of your MDP

The actions the company can take to affect the use status are adjusting or keeping the number of servers in the company's data center:

$A = \{\text{"down"}, \text{"stay"}, \text{"up"}, \text{"max"}\}$

|      |                                                                                                                                        |
|------|----------------------------------------------------------------------------------------------------------------------------------------|
| down | Scale-down by reducing the number of servers. Often adopted when the existing servers are underutilized.                               |
| stay | Keep the number of servers unchanged.                                                                                                  |
| up   | Scale-up by increasing the number of servers. Often adopted when the number of existing servers cannot satisfy the needs of customers. |
| max  | Maximize the number of servers. An extreme measure to build the largest cloud.                                                         |

According to the principles of MDP,  $P(s' | s, a)$  is the vector of transition probability for each state under each action  $a$ . A few dictionaries are provided in the starter code (mdp.py) to define these vectors. For the reward information  $R(s, a, s')$ , we define it as  $\text{Revenue}[s'] - \text{Cost}[a]$ . The Revenue and Cost tables and the reward function are also provided in the starter code.

You need to implement both value iteration and policy iteration methods to find the optimal policy for the company, i.e. the best action to take for each state. For the value iteration

method, you also need to find the maximum total expected reward for each state. These vectors must be printed by your program and analyzed in the task report.

In the starter code, the discount factor of the MDP is preset to 0.9 and the error parameter epsilon for the value iteration is preset to 0.1. You should first test the correctness of your functions with these values as the default setting. Think about how to verify the correctness, and if possible, turn your idea into testing code. Include your discovery in the task report.

For both algorithms, the number of iterations required to converge should be monitored. As you tune gamma and epsilon values, this number might be affected. Include your data and analyses in the task report as well.

As a special test, set your gamma  $\geq 0.99$  and compare the policy found by the two methods. Do they always agree with each other? If not, how often they disagree? If you dial the value of gamma down, how this adjustment changes the policies found by them? What are the potential problems caused by a gamma  $\geq 0.99$ ? Any mitigations?

### **Submission requirements**

1. Your program mdp.py

2. A report including the following points

- How you implemented each method? Any special data structures used? Specially in the policy iteration method, how you implemented the policy evaluation step? Did you solve the linear equation using linear algebra or approximation?
- Present and analyze the optimal policy and maximum total expected reward found by your methods
- Data and analyses about the correctness test
- Data and analyses about the number of iterations needed to converge under different settings of gamma and epsilon
- Data and analyses about the special test for gamma  $\geq 0.99$